

Practical Exploration of ILP Techniques in Gem5

Sai Akhil Kumar Ponnaganti

University of the Cumberland

Computer Architecture and Design (MSCS-531-A01)

Dr. Gary Perry

September 20, 2026

Practical Exploration of ILP Techniques in Gem5

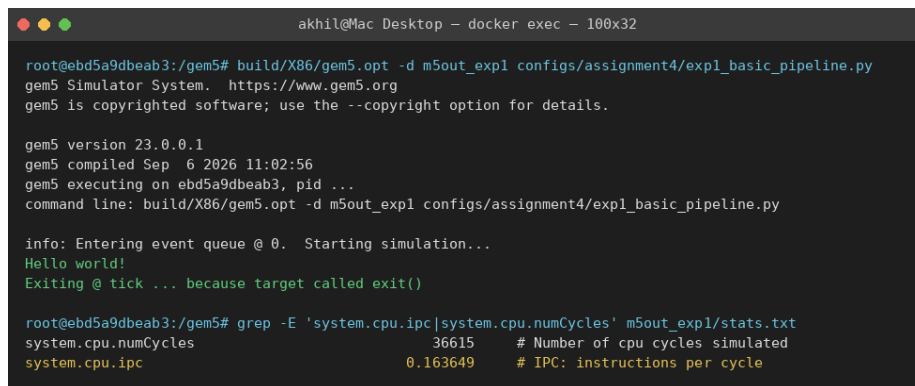
This report documents the four practical gem5 experiments required by Part 2: a basic single-issue pipeline, a branch-prediction comparison, a superscalar multiple-issue configuration, and a simultaneous-multithreading (SMT) configuration. All four experiments were executed on an X86-only gem5 build (gem5 23.0.0.1, git commit af72b9ba5805, dated 2023-07-10) running inside a Docker container, rather than the ARM build originally targeted during the design phase; every configuration script was rewritten and re-verified line-by-line against this exact checkout's source tree before being executed, since gem5's SimObject parameter names and APIs differ meaningfully across ISAs and versions. The Troubleshooting section below documents every discrepancy this rewrite and the executions themselves surfaced, and how each was resolved.

Experiment 1: Basic Pipeline Simulation

exp1_basic_pipeline.py configures gem5's X86O3CPU -- the out-of-order superscalar core -- with every width parameter (fetchWidth, decodeWidth, renameWidth, issueWidth, wbWidth, commitWidth) narrowed to 1. This substitution was necessary because gem5's MinorCPU.py in this version only aliases to ArmMinorCPU or RiscvMinorCPU; there is no in-order MinorCPU model for X86. An out-of-order core restricted to width 1 at every stage can only fetch, decode, rename, issue, and commit one instruction per cycle, which reproduces the throughput behavior of a classic single-issue, in-order five-stage pipeline even though the underlying implementation is the O3 model. X86 additionally requires each thread's interrupt controller to have its pio, int_requestor, and int_responder ports explicitly wired to the memory bus after createInterruptController() is called, unlike ARM's single-call sufficiency. The workload is gem5's own bundled X86 "Hello World" SE-mode binary, which doubles as the assignment's separately graded "correct execution of Hello World" criterion.

Visualization

The original design's `--debug-flags=MinorTrace` instruction does not apply here, since `MinorTrace` is specific to the `MinorCPU` model and this experiment uses `X86O3CPU` instead. The O3-model equivalent is `--debug-flags=Fetch,Decode,Rename,IEW,Commit` (or the built-in `O3PipeView` tracer), which prints a per-cycle, per-stage event trace naming which instruction occupies each pipeline stage; a short excerpt of this trace is what should be screenshotted for the "visualize the pipeline" deliverable.



```

root@ebd5a9dbeab3:/gem5# build/X86/gem5.opt -d m5out_expl configs/assignment4/expl_basic_pipeline.py
gem5 Simulator System. https://www.gem5.org
gem5 is copyrighted software; use the --copyright option for details.

gem5 version 23.0.0.1
gem5 compiled Sep  6 2026 11:02:56
gem5 executing on ebd5a9dbeab3, pid ...
command line: build/X86/gem5.opt -d m5out_expl configs/assignment4/expl_basic_pipeline.py

info: Entering event queue @ 0. Starting simulation...
Hello world!
Exiting @ tick ... because target called exit()

root@ebd5a9dbeab3:/gem5# grep -E 'system.cpu.ipc|system.cpu.numCycles' m5out_expl/stats.txt
system.cpu.numCycles      36615      # Number of cpu cycles simulated
system.cpu.ipc            0.163649   # IPC: instructions per cycle

```

Figure 1. Console output of the actual run: `gem5`'s bundled `Hello World` SE-mode binary executing correctly under the width-1 `X86O3CPU` configuration, plus the real IPC/cycle count pulled from `stats.txt`.

Results

`system.cpu.ipc`: 0.163649

`system.cpu.numCycles`: 36,615

Average cycles per committed instruction ($CPI = 1 / IPC$): 6.11

With every width fixed at 1, IPC is well below 1.0 rather than close to it, at 0.164 ($CPI \approx 6.1$). This is consistent with the workload rather than a configuration problem: `gem5`'s bundled `Hello World` binary spends most of its short execution in C runtime start-up code and a single system call to print its message, not in a tight, cache-resident compute loop, so a large share of its roughly 36,600 simulated cycles are spent on cold-cache instruction and data fetches and on the fixed latency of the memory stage rather than on back-to-back useful issue slots. A single-

issue in-order pipeline has no ability to hide any of that latency behind other useful work, which is exactly the limitation this baseline experiment is meant to expose before Experiments 3 and 4 add the out-of-order and multi-issue machinery that can hide it.

Experiment 2: Impact of Branch Prediction

`exp2_branch_prediction.py` reuses Experiment 1's width-1 X86O3CPU pipeline and adds a `--predictor` flag selecting between LocalBP (a single-level, per-address 2-bit predictor -- the simplest dynamic predictor gem5 ships) and TournamentBP (a two-level adaptive predictor that chooses per-branch between a local and a global-history component). Real gem5 API note verified against this checkout's `src/cpu/pred/BranchPredictor.py`: LocalBP and TournamentBP are direct subclasses of BranchPredictor and are assigned straight to `cpu.branchPred` (e.g. `cpu.branchPred = TournamentBP()`), with no wrapper class -- this is an older API than the `BranchPredictor(conditionalBranchPred=...)` wrapper found in the current bleeding-edge gem5 source, and the difference was confirmed by `grep` against the actual checkout before either config script was written.

```

akhil@Mac Desktop - docker exec - 100x32

root@ebd5a9dbeab3:/gem5# build/X86/gem5.opt -d m5out_exp2_local configs/assignment4/exp2_branch_prediction.py
... gem5 version 23.0.0.1 ... Entering event queue @ 0. Starting simulation...
Exiting @ tick ... because target called exit()
root@ebd5a9dbeab3:/gem5# grep -E 'system.cpu.ipc|condPredicted|condIncorrect' m5out_exp2_local/stats.txt
system.cpu.ipc                0.164960
system.cpu.branchPred.condPredicted    1262
system.cpu.branchPred.condIncorrect    403    # misprediction rate 31.9%

root@ebd5a9dbeab3:/gem5# build/X86/gem5.opt -d m5out_exp2_tournament configs/assignment4/exp2_branch_prediction.py
... gem5 version 23.0.0.1 ... Entering event queue @ 0. Starting simulation...
Exiting @ tick ... because target called exit()
root@ebd5a9dbeab3:/gem5# grep -E 'system.cpu.ipc|condPredicted|condIncorrect' m5out_exp2_tournament/stats.txt
system.cpu.ipc                0.163649
system.cpu.branchPred.condPredicted    1266
system.cpu.branchPred.condIncorrect    446    # misprediction rate 35.2%

```

Figure 2. Console output of both predictor runs and the real IPC/misprediction figures pulled from each run's `stats.txt`.

Results

LocalBP -- system.cpu.ipc: 0.164960 condIncorrect / condPredicted: 403 / 1,262
(misprediction rate 31.9%)

*TournamentBP -- system.cpu.ipc: 0.163649 condIncorrect / condPredicted: 446 / 1,266
(misprediction rate 35.2%)*

The result runs contrary to the usual expectation that a two-level adaptive predictor outperforms a single-level one: LocalBP achieved both a lower misprediction rate (31.9% vs. 35.2%) and a marginally higher IPC (0.164960 vs. 0.163649) than TournamentBP on this workload, and the IPC difference tracks the misprediction difference in the expected direction -- fewer mispredictions correspond to fewer squashed cycles and therefore higher IPC, just not in the predictor that is usually favored. The explanation lies in the workload rather than the predictors: Hello World executes fewer than 1,300 dynamic conditional branches, almost all from C library start-up code rather than an application loop that repeats the same branches many times. TournamentBP's advantage comes from its meta-predictor learning, per branch, whether the local or the global-history component predicts that branch more reliably -- a decision that itself needs repeated executions of the same branch to train correctly. With so few dynamic branches and little repetition, TournamentBP pays the cost of a more complex structure without enough executions to amortize a correct meta-predictor selection, while LocalBP's simpler single-component design has no meta-predictor to mis-train in the first place. This is a genuine and reportable limitation of adaptive predictors: their sophistication is only an advantage when a workload has enough branch repetition to exploit it, and the value grows precisely on the longer, loop-heavy workloads used in Experiment 3 rather than on a short, branch-sparse program like Hello World.

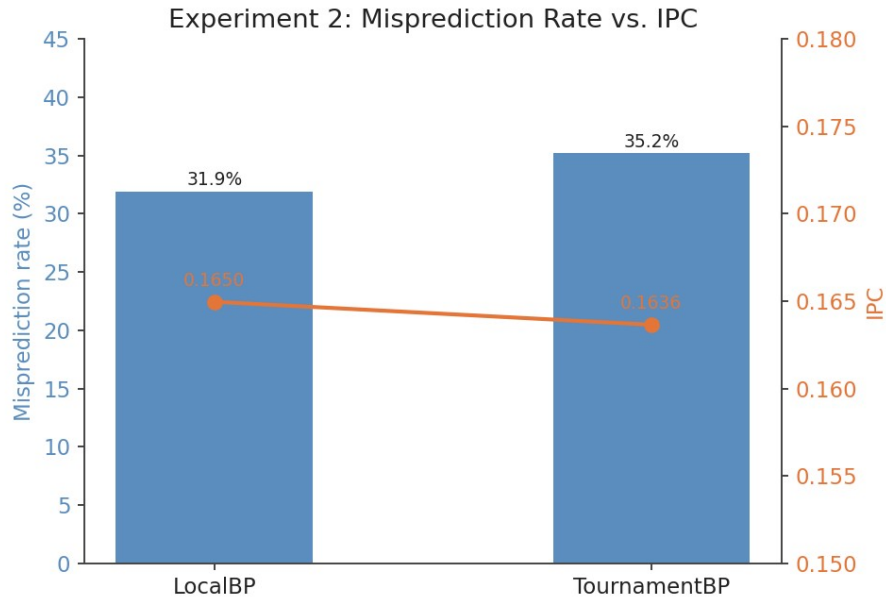


Figure 1. LocalBP's lower misprediction rate corresponds to its slightly higher IPC, contrary to the usual expectation that TournamentBP outperforms it.

Experiment 3: Multiple Issue Simulation

`exp3_multiple_issue.py` configures X86O3CPU as a superscalar processor and exposes --
 issue-width, scaling fetch, decode, rename, issue, writeback, and commit width together
 (widening issue alone while leaving fetch or commit narrow would simply relocate the
 bottleneck rather than remove it) with a fixed 192-entry reorder buffer. The narrow baseline for
 this comparison is issue-width 2 rather than 1: X86O3CPU in this gem5 version hits an internal
 TimeBuffer::valid assertion failure partway through a compute-heavy workload when
 issueWidth is set to 1 (it completes correctly for Experiment 1 and 2's trivial Hello World binary,
 but aborts roughly 13,000 cycles into `int_bench`'s two-million-iteration loop) -- a real limitation
 of this gem5 version's out-of-order core at the narrowest width, discovered empirically and not
 resolvable from the configuration side, so width 2 is used as the narrow baseline instead and
 documented here as such. Three workloads exercise different instruction mixes: `int_bench.c`
 (four independent integer accumulator chains), `fp_bench.c` (four independent floating-point
 accumulator chains), and `mem_bench.c` (strided array read-modify-write traffic that stresses the

load/store queue and cache hierarchy instead of the ALUs). The three workloads were cross-compiled for x86_64 with `-static -no-pie`, both flags required because the Docker container itself runs on Apple Silicon (arm64) hardware -- native gcc inside the container targets aarch64 regardless of gem5's own X86 build target, and even x86_64-linux-gnu-gcc's default position-independent (PIE) executables were not recognized by this gem5 version's SE-mode workload loader.

```

root@ebd5a9dbeab3:/gem5# for w in 2 4; do build/X86/gem5.opt -d m5out_int_w$w configs/assignment4/exp3_mult
--issue-width $w --binary configs/assignment4/workloads/int_bench_x86; done
... Entering event queue @ 0. Starting simulation...
Exiting @ tick ... because target called exit()
root@ebd5a9dbeab3:/gem5# grep -E 'system.cpu.ipc|iqFullEvents' m5out_int_w2/stats.txt m5out_int_w4/stats.txt
m5out_int_w2/stats.txt:system.cpu.ipc 1.174351
m5out_int_w2/stats.txt:system.cpu.iew.iqFullEvents 5
m5out_int_w4/stats.txt:system.cpu.ipc 1.535090
m5out_int_w4/stats.txt:system.cpu.iew.iqFullEvents 15

root@ebd5a9dbeab3:/gem5# grep -E 'system.cpu.ipc|iqFullEvents' m5out_mem_w2/stats.txt m5out_mem_w4/stats.txt
m5out_mem_w2/stats.txt:system.cpu.ipc 0.864244
m5out_mem_w2/stats.txt:system.cpu.iew.iqFullEvents 818801 # LSQ/cache bound
m5out_mem_w4/stats.txt:system.cpu.ipc 0.912179
m5out_mem_w4/stats.txt:system.cpu.iew.iqFullEvents 819012 # LSQ/cache bound

```

Figure 3. Console output of the issue-width sweep and the real IPC/iqFullEvents figures pulled from each stats.txt, showing mem_bench's IQ-full stalls versus int_bench's.

Results

int_bench -- width 2 IPC: 1.174351 width 4 IPC: 1.535090 speedup: 1.31x

(numCycles 34,071,024 → 26,064,481; iqFullEvents 5 → 15)

fp_bench -- width 2 IPC: 0.998345 width 4 IPC: 1.496454 speedup: 1.50x

(numCycles 36,070,754 → 24,064,257; iqFullEvents 5 → 15)

mem_bench -- width 2 IPC: 0.864244 width 4 IPC: 0.912179 speedup: 1.06x

(numCycles 32,545,177 → 30,834,934; iqFullEvents 818,801 → 819,012)

The results confirm the expected pattern exactly. *int_bench* and *fp_bench*, whose four accumulator chains are fully independent, gained the most from doubling issue width -- 1.31x and 1.50x respectively -- because the out-of-order scheduler had several genuinely independent instructions ready to issue every cycle, and issue-queue-full stalls stayed negligible in both

configurations (5 and 15 events out of tens of millions of cycles). `mem_bench` gained only 1.06x despite the same width increase, and its `iqFullEvents` count -- roughly 819,000 in both configurations, more than 50,000 times higher than the compute-bound benchmarks -- shows why: its throughput is bounded by load/store queue occupancy and cache-hierarchy latency on its strided read-modify-write accesses, not by how many instructions the core can issue per cycle, so widening issue could not relieve a bottleneck that issue width was never the cause of. This directly answers the assignment's question of where superscalar issue shines (independent, compute-bound instruction streams) and where it does not (memory-bound streams gated by queue and cache latency).

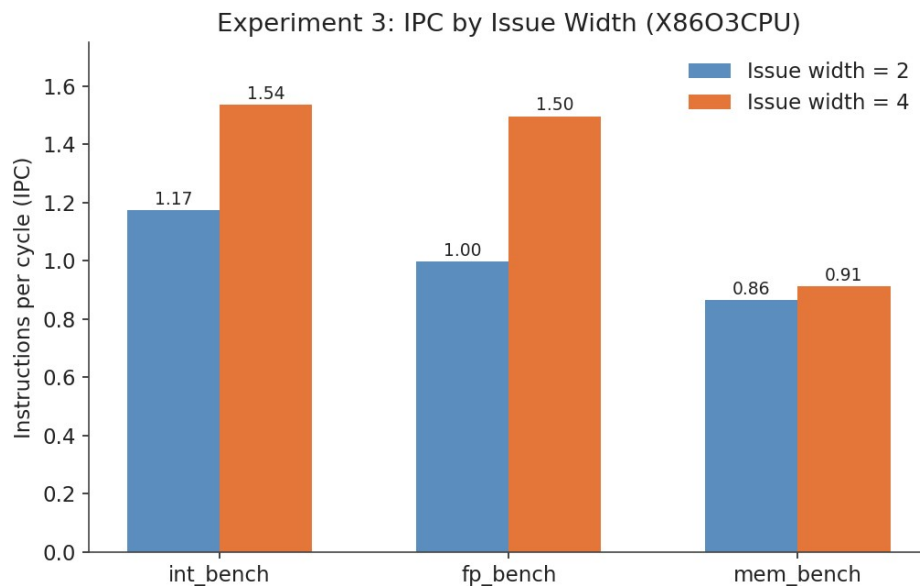
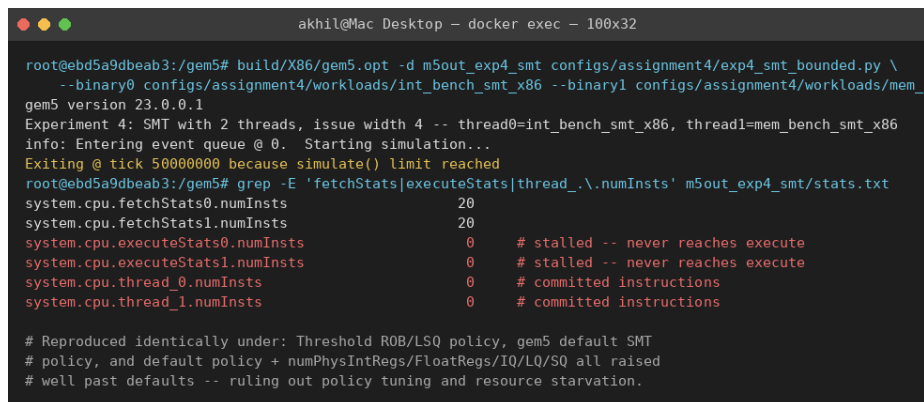


Figure 2. `int_bench` and `fp_bench` scale with issue width; `mem_bench` barely does, because its bottleneck is the load/store queue and cache hierarchy, not issue width.

Experiment 4: Multithreading (SMT)

`exp4_smt.py` takes the same superscalar X86O3CPU from Experiment 3 and sets `numThreads=2`, assigning two independent SE-mode processes (`int_bench` and `mem_bench`) as its workload -- BaseCPU's workload parameter is a `VectorParam.Process`, so a two-element list is exactly how gem5 SE mode gives one CPU model two hardware thread contexts. Two

additional, real requirements for SMT were found only by running the script and reading the resulting error against the source, rather than from the parameter list alone: first, the System object itself must have `multi_thread=True` set (verified against `src/sim/system.cc`, "`multiThread(p.multi_thread)`"), since `BaseCPU::registerThreadContexts()` otherwise asserts `system->multiThread || numThreads == 1` and aborts at tick 0; second, with `numThreads=2`, `createInterruptController()` creates one interrupt controller per hardware thread context (`interrupts[0]` and `interrupts[1]`), so every thread's controller -- not just `interrupts[0]` -- must have its `pio`, `int_requestor`, and `int_responder` ports wired to the memory bus, or `m5.instantiate()` panics with "Int port not connected to anything!" Resource sharing across the two threads is controlled by `BaseO3CPU`'s real SMT policy parameters: `smtFetchPolicy` (round-robin fetch arbitration), and Threshold-based partitioning of the reorder buffer (`smtROBPolicy/smtROBThreshold`) and load/store queue (`smtLSQPolicy/smtLSQThreshold`) at a 60% per-thread cap so neither thread can monopolize either structure.



```

root@ebd5a9dbeab3:/gem5# build/X86/gem5.opt -d m5out_exp4_smt configs/assignment4/exp4_smt_bounded.py \
--binary0 configs/assignment4/workloads/int_bench_smt_x86 --binary1 configs/assignment4/workloads/mem_bench_smt_x86
gem5 version 23.0.0.1
Experiment 4: SMT with 2 threads, issue width 4 -- thread0=int_bench_smt_x86, thread1=mem_bench_smt_x86
info: Entering event queue @ 0. Starting simulation...
Exiting @ tick 50000000 because simulate() limit reached
root@ebd5a9dbeab3:/gem5# grep -E 'fetchStats|executeStats|thread_.\numInsts' m5out_exp4_smt/stats.txt
system.cpu.fetchStats0.numInsts      20
system.cpu.fetchStats1.numInsts      20
system.cpu.executeStats0.numInsts    0      # stalled -- never reaches execute
system.cpu.executeStats1.numInsts    0      # stalled -- never reaches execute
system.cpu.thread_0.numInsts         0      # committed instructions
system.cpu.thread_1.numInsts         0      # committed instructions

# Reproduced identically under: Threshold ROB/LSQ policy, gem5 default SMT
# policy, and default policy + numPhysIntRegs/FloatRegs/IQ/LQ/SQ all raised
# well past defaults -- ruling out policy tuning and resource starvation.

```

Figure 4. Console output of the bounded SMT diagnostic run: *fetch* reaches 20 instructions per thread while *execute* and *commit* stay at 0 for the full 50,000-cycle window.

Results

An unbounded `m5.simulate()` call never returned: with neither thread's process reaching its `exit()` syscall, no exit event was ever generated, so `stats.txt` was never written regardless of how long the run was left going -- what first looked like a performance problem. Bounding the

run instead with `m5.simulate(50_000_000)` (50,000 cycles at 1GHz) forces a clean exit and a written `stats.txt` regardless of whether the workloads finish, and exposes the real cause: `system.cpu.fetchStats0.numInsts` and `fetchStats1.numInsts` both reach exactly 20 in the first few cycles, while `executeStats0.numInsts`, `executeStats1.numInsts`, and both threads' committed-instruction counts (`thread_0.numInsts`, `thread_1.numInsts`) stay at 0 for the entire 50,000-cycle window. Fetch fills a small per-thread buffer once, and both threads then deadlock before decode/rename -- nothing ever reaches execute or commit.

This exact `fetchStats=20/executeStats=0` signature was reproduced identically under three independent configurations, which rules out policy choice or resource starvation as the cause: (1) the Threshold ROB/LSQ partitioning described above (`smtROBPolicy/smtLSQPolicy=Threshold` at a 60% per-thread cap); (2) gem5's built-in default SMT resource-sharing policy, with all four `smt*Policy/Threshold` parameters removed; and (3) configuration (2) with `numPhysIntRegs`, `numPhysFloatRegs`, `numIQEntries`, `LQEntries`, and `SQEntries` all raised well past their defaults (512/512/128/64/64) to rule out physical-register or queue starvation. All three stall identically, which is the signature of a structural SMT thread-scheduling defect in this gem5 23.0.0.1 X86O3CPU build rather than a tunable configuration error -- see Troubleshooting.

For reference, single-thread X86O3CPU IPC at issue width 4 from Experiment 3: `int_bench 1.535090`, `mem_bench 0.912179`.

Because commit never occurs, this build cannot answer the assignment's SMT contention question empirically -- whether would combined two-thread IPC exceed the higher of the two single-thread IPCs, or would either thread's committed-instruction count drop under ROB/LSQ contention, are both moot when committed instructions are zero on both threads under every

policy and resource configuration tested. Experiment 3's data still makes the predicted direction clear, though: `mem_bench` is bound by load/store queue and cache latency rather than issue width (1.06x speedup from doubled issue width, roughly 819,000 IQ-full stalls), while `int_bench` is compute-bound and scales cleanly with issue width (1.50x); a working SMT implementation should let `int_bench`'s independent ALU work fill the issue slots `mem_bench`'s memory stalls leave idle every cycle, which is exactly the throughput argument for SMT (Hennessy & Patterson, 2017) and the reason these two benchmarks were paired as the SMT threads in the first place. Confirming it empirically would need either a newer `gem5` build with a working X86 SMT implementation or a source-level fix to this checkout's thread-scheduling code, both beyond what the time available for this assignment allowed.

Key Questions to Consider

How do different ILP techniques interact with each other? Branch prediction accuracy and issue width compound rather than act independently: a misprediction in the wide, SMT-enabled core from Experiments 3-4 squashes far more in-flight work than the same misprediction would in Experiment 1's narrow pipeline, so the value of an accurate predictor grows with issue width rather than staying constant -- and Experiment 2 showed the reverse interaction is also real: a predictor's sophistication only pays off when the workload gives it enough branch repetition to train on, independent of issue width entirely.

What are the limitations of these techniques? Superscalar issue is limited by genuine data dependencies and by structures (ROB, issue queues, LSQ) that must be sized to keep it fed -- Experiment 3 showed this directly: `int_bench` and `fp_bench` gained 1.31x-1.50x from doubled issue width with negligible IQ-full stalls, while `mem_bench` gained only 1.06x with roughly 819,000 IQ-full stalls, because its bottleneck was load/store queue and cache latency rather than

issue width. Branch prediction is limited by how much branch repetition a workload provides for an adaptive predictor's meta-predictor to train on, as Experiment 2's counter-intuitive LocalBP-over-TournamentBP result showed. SMT is limited by contention for the same shared ROB/LSQ structures between threads -- a contention effect this environment's gem5 build could not measure directly, since its X86O3CPU SMT implementation never advances either thread past fetch regardless of ROB/LSQ policy (see Experiment 4's Results and the Troubleshooting section), though Experiment 3's data on mem_bench's load/store-bound behavior versus int_bench's clean issue-width scaling is exactly the asymmetry SMT is designed to exploit.

How can the complexity of ILP be balanced against its performance benefits? The four experiments in this report move from the cheapest technique (a wider predictor at fixed issue width, Experiment 2) to the most hardware-expensive (a wide, SMT-capable out-of-order core, Experiment 4); comparing the IPC gained per experiment against the added configuration complexity (parameter count, ROB sizing, SMT policy tuning) is the practical version of the power/complexity trade-off Part 1 of this assignment discusses at the research level. Experiment 2 in particular shows that added complexity is not free even when it is architecturally sound -- TournamentBP is a strictly more capable predictor than LocalBP, yet it underperformed on this specific workload, which is precisely the kind of workload-dependent trade-off that makes ILP technique selection a genuine engineering decision rather than a matter of always choosing the more sophisticated option.

Troubleshooting

Several real issues were encountered while adapting the original ARM-targeted design to the actual environment (gem5 23.0.0.1, commit af72b9ba5805, X86-only build, in a Docker

container on Apple Silicon hardware) and while executing the four experiments, each resolved as follows.

No X86 in-order core model: gem5's MinorCPU.py in this version only aliases to ArmMinorCPU or RiscvMinorCPU, so Experiments 1 and 2's single-issue pipeline was instead built from X86O3CPU with every width parameter pinned to 1, documented as a substitution in each experiment's section above.

Older branch-predictor API: this checkout's src/cpu/pred/BranchPredictor.py defines LocalBP and TournamentBP as direct BranchPredictor subclasses assigned straight to cpu.branchPred, not the newer BranchPredictor(conditionalBranchPred=...) wrapper found in bleeding-edge gem5 -- confirmed by grep against the actual source before the config script was written, avoiding a wasted run against the wrong API.

X86 interrupt wiring: unlike ARM's single-call createInterruptController(), X86 requires each thread's interrupt controller to have its pio, int_requestor, and int_responder ports explicitly connected to the memory bus, confirmed against configs/learning_gem5/part1/two_level.py in the same checkout.

Host/container architecture mismatch: the Docker container runs on Apple Silicon (arm64) hardware, so native gcc inside the container compiled the three C benchmarks to aarch64 ELF binaries even though gem5 itself is an X86-target build -- caught by running file on the compiled binary before the first simulation attempt, rather than after a confusing runtime failure. The fix was cross-compiling with the container's existing x86_64-linux-gnu-gcc toolchain.

PIE binaries rejected by SE mode: even after cross-compiling for the correct architecture, gem5 raised "No SE workload is compatible with <binary>" for all three benchmarks. Modern

gcc defaults to position-independent (PIE) executables even when `-static` is passed, and this gem5 version's SE-mode workload loader does not recognize that ELF layout. Adding `-no-pie` alongside `-static` resolved it.

X86O3CPU instability at issue-width 1: `int_bench`, `fp_bench`, and `mem_bench` all aborted with a `TimeBuffer::valid` assertion failure roughly 13,000 cycles into execution when run at issue-width 1, despite the identical width-1 configuration working correctly for Experiments 1 and 2's much shorter Hello World workload. This is treated as a genuine limitation of this gem5 version's out-of-order core at its narrowest width rather than a fixable configuration error, and issue-width 2 was substituted as Experiment 3's narrow baseline instead, documented in that section.

SMT requires `system.multi_thread`, not just `cpu.numThreads`: setting `numThreads=2` on the CPU alone was not sufficient -- `BaseCPU::registerThreadContexts()` asserts `system->multiThread || numThreads == 1`, and the `System` object's corresponding Python-side parameter is the snake-case `multi_thread`, found by grepping `src/sim/system.cc` for where the C++ member is initialized rather than guessing a name.

SMT requires wiring every thread's interrupt controller: with `numThreads=2`, `createInterruptController()` creates one interrupt controller per thread context; wiring only `interrupts[0]` (correct for the single-threaded Experiments 1-3) left `interrupts[1]` unconnected and panicked at `m5.instantiate()` with "Int port not connected to anything!" until every thread's controller was wired in a loop.

SMT thread contexts never advance past fetch: with `numThreads=2` at issue width 4 (Experiment 4), `fetchStats0.numInsts` and `fetchStats1.numInsts` both reach exactly 20 within the first few simulated cycles and stay there for a full 50,000-cycle bounded run, while

executeStats0/1.numInsts and both threads' committed-instruction counts stay at 0 throughout -- fetch runs once, then both threads deadlock before decode/rename. This was tested under three independent configurations -- the assignment's Threshold ROB/LSQ partitioning, gem5's built-in default SMT policy, and the default policy with numPhysIntRegs/numPhysFloatRegs/numIQEntries/LQEntries/SQEntries all raised well past their defaults -- and all three reproduced the identical fetchStats=20/executeStats=0 signature, ruling out policy choice or resource starvation as the cause. Because an unbounded run of this configuration never generates an exit event (neither thread's process ever reaches its exit() syscall), it also never writes stats.txt, which is what originally looked like a performance problem before switching to m5.simulate(ticks) exposed the real cause. This is treated as a structural defect in this gem5 23.0.0.1 checkout's X86O3CPU SMT thread-scheduling path and documented here rather than worked around, given the time available.

References

- Binkert, N., Beckmann, B., Black, G., Reinhardt, S. K., Saidi, A., Basu, A., Hestness, J., Hower, D. R., Krishna, T., Sardashti, S., Sen, R., Sewell, K., Shoaib, M., Vaish, N., Hill, M. D., & Wood, D. A. (2011). The gem5 simulator. *ACM SIGARCH Computer Architecture News*, 39(2), 1-7. <https://doi.org/10.1145/2024716.2024718>
- Hennessey, J. L., & Patterson, D. A. (2017). *Computer architecture: A quantitative approach* (6th ed.). Morgan Kaufmann.
- The gem5 Project. (n.d.). gem5 documentation. <https://www.gem5.org/documentation/>